

Group 1

L03 ML Workflow & Learning Types

ITAI-1371

Viswanatha Rao

Reflection Journal - ML Workflow and Learning Types

Before this lab, our group mostly thought about machine learning as a process of choosing an algorithm, training it, and checking the final accuracy. After working through the full Wine Classification project and comparing our observations, that view became much more detailed. We began to understand machine learning as a connected process where each decision can affect the next step. Data preparation, feature selection, splitting the dataset, training, evaluation, and interpretation all work together rather than as separate tasks. This was probably the biggest change in our thinking. Instead of asking only which model performs best, we began asking what data the model learned from, how the data was divided, which features were used, what kinds of mistakes the model made, and whether the result would still make sense outside one test run.

The differences between supervised, unsupervised, and reinforcement learning became easier to understand once we connected each type to a real problem. Supervised learning made the most sense in the Wine Classification project because the dataset already included known wine classes. The model had an answer key during training, so it could learn the relationship between the selected features and the correct class labels. Unsupervised learning became interesting for a different reason. Without labels, the model has to search for patterns, similarities, or unusual groups on its own. Reinforcement learning follows another approach because the model learns from the results of its actions through rewards and penalties. Looking at these examples helped our group understand that the best type of learning depends on the problem and the available data, not on which method sounds the most advanced.

The most challenging part of the workflow was understanding model performance beyond a single accuracy score. At first, comparing 88.9% for Logistic Regression with 83.3% for the Decision Tree seemed simple. Logistic Regression had the higher percentage, so it looked like the clear winner. However, the classification report and confusion matrix made us look more closely at the results. They showed that model performance can differ across classes and that overall accuracy does not explain where the mistakes happen. This changed our approach to evaluation because we stopped treating one number as the whole result. A model can perform well overall while still struggling with one class, and that weakness could become important in a real-world situation.

The train-and-test split also became more meaningful during the lab. We used 80% of the data for training and 20% for testing, while stratification helped keep the class proportions similar. Before this lab, splitting the data felt like just another coding step we had to complete before training the model. Now we understand why it matters. If we judge the model using only the examples it already saw, a high score does not necessarily show that it learned patterns that will work on new data. Testing it on unseen examples gives us a better idea of how well it can generalize. This also helped make the idea of overfitting more practical because we could see why memorizing the training data is not the same as learning something useful.

The feature experiment gave us one of the strongest examples of how earlier workflow choices can affect the final result. The original Logistic Regression model used alcohol, malic acid, ash, and

alcalinity of ash and reached 88.9% accuracy. We later tested flavanoids, color intensity, and proline while keeping the same model type, and the accuracy increased to 91.7%. What stood out was that the algorithm stayed the same while the information given to the model changed. This showed us that feature selection can have a strong effect on performance, sometimes just as much as the choice of model. At the same time, we would not assume that 91.7% will always be better based on one split. A useful next step would be cross-validation so we could test whether the improvement remains consistent across different splits. We would also want to test scaling and different model settings instead of assuming that one change solved everything.

This lab also connected the current workflow to the tools we used in the previous lab. Pandas, NumPy, and visualization started to feel less like separate practice skills and more like parts of the same machine learning process. Looking at the class distribution helped us check whether the dataset was balanced, while the correlation heatmap helped us examine relationships among features. These steps gave us information that could guide later choices. That connection helped us understand that exploratory data analysis is not something we finish before the 'real' machine learning begins. It plays an important role throughout the process by helping us decide what to try next and understand the results we get later.

Cybersecurity gave our group a useful way to think about applying these ideas outside the Wine dataset. A machine learning system could analyze login times, failed login attempts, IP addresses, device information, network traffic, user activity patterns, previous alerts, and confirmed attacks. Unsupervised learning could be especially useful when the data does not contain labels for every possible threat. A model could learn common behavior patterns and flag unusual activity for closer review. However, our discussion also made the limitations clear. Unusual activity does not always mean an attack, so false positives could waste time or affect normal users. A real threat could also look normal enough to avoid detection. For that reason, we would not treat the model as a replacement for human review. Instead, it could help security teams focus attention on the most unusual or risky activity.

This lab also brought up several questions that our group would like to explore further. We want to understand how machine learning systems can identify threats they have never seen before, how developers reduce false positives without increasing the risk of missing real attacks, and how often a model should be retrained when behavior and attack patterns continue to change. The feature experiment also made us curious about how feature selection works with much larger datasets that may contain hundreds or even thousands of variables. These questions showed us that evaluating one finished model is only part of the process. Each result can lead to a new question, a new idea, or another experiment.

Overall, this lab changed how our group looks at machine learning. We no longer see the workflow as a straight path from data to a final prediction. Each stage can influence the next, and evaluation can send us back to reconsider earlier choices. The improvement from 88.9% to 91.7% after changing the feature set made that idea especially clear, while the classification report and confusion matrix reminded us that a higher score does not tell the whole story. Our main takeaway is that machine learning requires experimentation, careful evaluation, and a willingness to question the result instead of accepting it immediately. Going forward, we would pay more attention to the data the model learns from, how we evaluate it, where it makes mistakes, and whether its performance would still be useful in a real-world situation.